

Large Language Model

Shihua Zhang

January 2, 2025

Outline

- 1 Introduction to Large Language Model
- 2 Scaling Laws and Emergence
- 3 Grokking
- 4 Chain of Thought
- 5 Advanced Reasoning Model: OpenAI's o1 and o3

Language Modeling and its Evolution

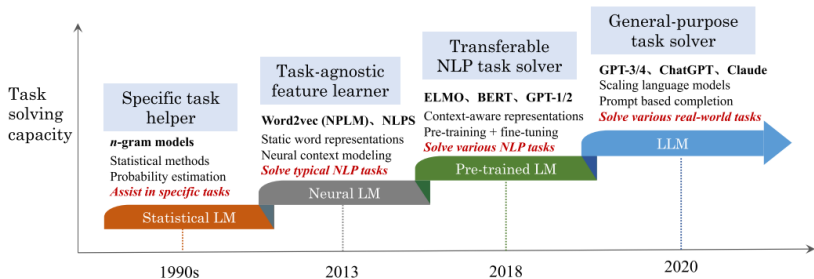
- **Language** is a key ability to express and communicate, developing from early childhood and lasting a lifetime.
- Machines require advanced **AI algorithms** to understand and generate natural language like humans.

Language Modeling and its Evolution

- **Language** is a key ability to express and communicate, developing from early childhood and lasting a lifetime.
- Machines require advanced **AI algorithms** to understand and generate natural language like humans.
- **Language Modeling (LM)** models the generative likelihood of word sequences and predict **probabilities** of words in context.

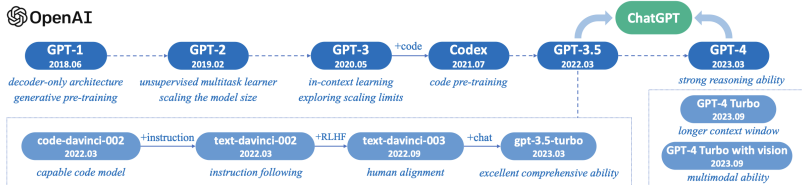
Language Modeling and its Evolution

- **Language** is a key ability to express and communicate, developing from early childhood and lasting a lifetime.
- Machines require advanced **AI algorithms** to understand and generate natural language like humans.
- **Language Modeling (LM)** models the generative likelihood of word sequences and predict **probabilities** of words in context.
- Development Timeline:



Overview of LLM Training Process

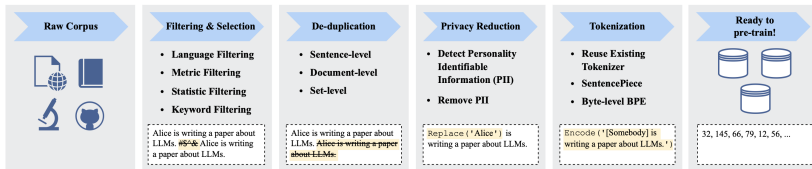
- **Pretraining**: Learn general language features from large-scale unlabeled data
- **Fine-tuning**: Optimize the model for specific tasks
- **RLHF**: Reinforcement Learning from Human Feedback



Pretraining Data Processing

Pretraining data is processed through steps like

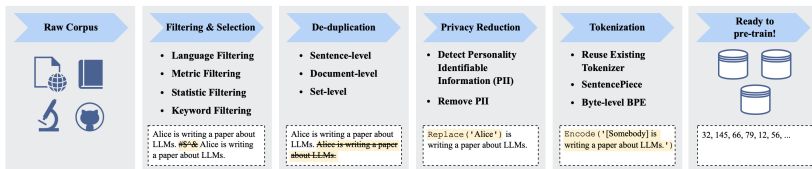
- Quality Filtering
- De-Duplication
- Privacy Reduction
- Tokenization



Pretraining Data Processing

Pretraining data is processed through steps like

- Quality Filtering
- De-Duplication
- Privacy Reduction
- Tokenization

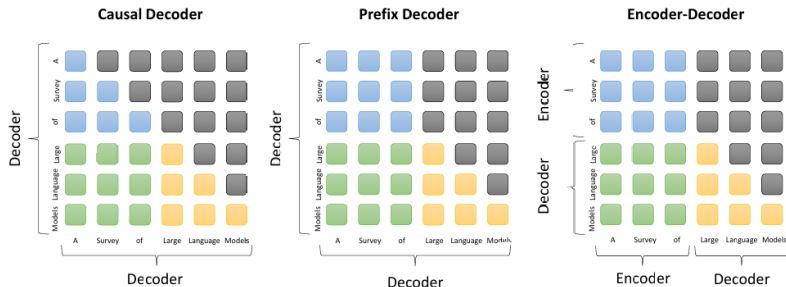


- Let $\{x_1, x_2, \dots, x_T\}$ represent the input sequence of words:

$$\min L = -\frac{1}{T} \sum_{t=1}^T \log p_{\theta}(x_t | x_1, \dots, x_{t-1})$$

Three LLM Architectures

- **Encoder-Decoder**: Utilizes both an encoder and a decoder for full context understanding and text generation (e.g., [Vanilla Transformer model](#)).
- **Causal Decoder**: Uses only a decoder to generate text with unidirectional dependencies (e.g., [GPT-series](#)).
- **Prefix Decoder**: Incorporates partial encoded information in the decoder for partial context awareness (e.g., [PaLM](#)).



Instruction Fine-Tuning (Supervised Learning)

- Collect demonstration data and train a supervised policy.

Instruction Fine-Tuning (Supervised Learning)

- Collect demonstration data and train a supervised policy.
- A **prompt** is sampled from prompt dataset.
 - For example, how to maintain abundant energy?
- A labeler demonstrate the desired **response** behavior.
 - For example, one cup of coffee every day.
- The data $\langle \text{prompt}, \text{response} \rangle$ is used to fine-tune GPT with supervised learning.

Instruction Fine-Tuning (Supervised Learning)

- Collect demonstration data and train a supervised policy.
- A **prompt** is sampled from prompt dataset.
 - For example, how to maintain abundant energy?
- A labeler demonstrate the desired **response** behavior.
 - For example, one cup of coffee every day.
- The data $\langle \text{prompt}, \text{response} \rangle$ is used to fine-tune GPT with supervised learning.
- Two general methods for constructing instruction datasets [Zhang et al., 2023]:
 - Data integration from annotated natural language datasets.
 - Generating outputs using LLMs.

RLHF (Imitating Human Feedback)

- Sample prompts from the prompt dataset and generate k responses using the fine-tuned model.
- Labeler **rank multiple responses** to the same prompt without scoring them.
- Train a reward model

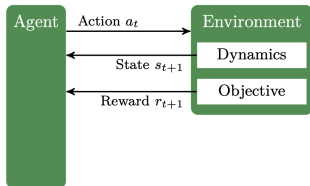
$$\text{loss}(\theta) = -\frac{1}{C_k^2} \mathbb{E}_{(x, y_w, y_l) \sim D} [\log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l)))].$$

RLHF (Imitating Human Feedback)

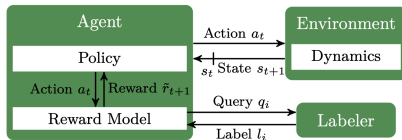
- Sample prompts from the prompt dataset and generate k responses using the fine-tuned model.
- Labeler **rank multiple responses** to the same prompt without scoring them.
- Train a reward model

$$\text{loss}(\theta) = -\frac{1}{C_k^2} \mathbb{E}_{(x, y_w, y_l) \sim D} [\log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l)))].$$

- Use the reward model to imitate human feedback and **give rewards** to response generated by LLM.



(a) The standard RL setting.



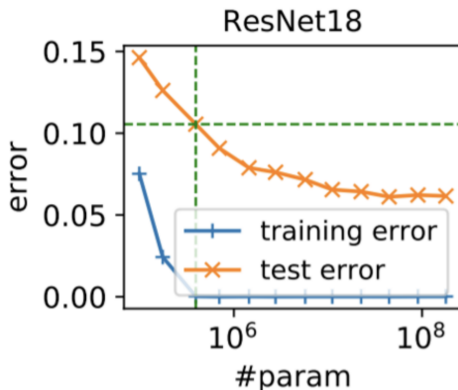
(b) RLHF with reward modeling

Outline

- 1 Introduction to Large Language Model
- 2 Scaling Laws and Emergence**
- 3 Grokking
- 4 Chain of Thought
- 5 Advanced Reasoning Model: OpenAI's o1 and o3

Over-parameterized DNNs

- DNNs are often highly **over-parameterized**.
- Test error **continues to decrease for larger networks**, though which are large enough to fit the training data [Neyshabur et al., 2018].



Scaling Laws for Transformer Language Models

- Language provides a natural domain for the study of AI.
- All texts provide a wealth of data for **unsupervised learning**.

Scaling Laws for Transformer Language Models

- Language provides a natural domain for the study of AI.
- All texts provide a wealth of data for **unsupervised learning**.
- Language modeling performance improves **smoothly** as we increase the model size.

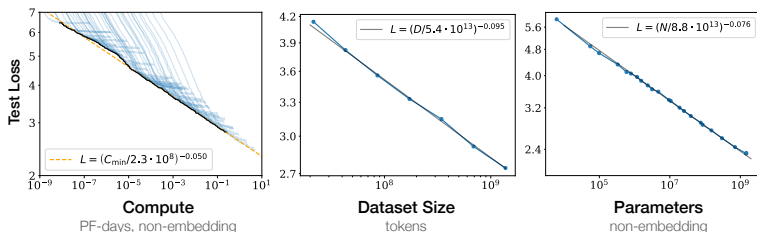


Figure: Empirical performance has a power-law relationship with each individual factor (Computing, Data, Parameters).

KM Scaling Law

- The KM scaling law describes the power-law relationship between model performance and three factors [OpenAI, 2020]:
 - **Model size (N)**: The number of model parameters.
 - **Dataset size (D)**: The total amount of training data.
 - **Training compute (C)**: The computational resources required.

KM Scaling Law

- The KM scaling law describes the power-law relationship between model performance and three factors [OpenAI, 2020]:
 - **Model size (N)**: The number of model parameters.
 - **Dataset size (D)**: The total amount of training data.
 - **Training compute (C)**: The computational resources required.
- The formulas are as follows:

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D}, \quad L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C} \quad (1)$$

- Here, α_N , α_D , α_C are the scaling exponents.
- $\alpha_N \sim 0.076$, $\alpha_D \sim 0.095$, $\alpha_C \sim 0.050$.

Chinchilla Scaling Law

- The Chinchilla scaling law improves the allocation of compute budgets [Hoffmann et al., 2022].
- Formula:

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad (2)$$

where $E = 1.69$, $A = 406.4$, $B = 410.7$, $\alpha = 0.34$, $\beta = 0.28$.

- Under the optimization condition $C \approx 6ND$, the optimal allocation:

$$N_{opt}(C) = G \left(\frac{C}{6} \right)^a, \quad D_{opt}(C) = G^{-1} \left(\frac{C}{6} \right)^b, \quad (3)$$

where $a = \frac{\alpha}{\alpha + \beta}$, $b = \frac{\beta}{\alpha + \beta}$.

Chinchilla Scaling Law

- The Chinchilla scaling law improves the allocation of compute budgets [Hoffmann et al., 2022].
- Formula:

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad (2)$$

where $E = 1.69$, $A = 406.4$, $B = 410.7$, $\alpha = 0.34$, $\beta = 0.28$.

- Under the optimization condition $C \approx 6ND$, the optimal allocation:

$$N_{opt}(C) = G \left(\frac{C}{6} \right)^a, \quad D_{opt}(C) = G^{-1} \left(\frac{C}{6} \right)^b, \quad (3)$$

where $a = \frac{\alpha}{\alpha + \beta}$, $b = \frac{\beta}{\alpha + \beta}$.

- The two sizes (N & D) should be increased **in equal scales**.

Explanation of Chinchilla Scaling Law

- From the equations, given an increase in compute budget:
 - The KM scaling law favors a larger budget allocation to model size (N).
 - The Chinchilla scaling law argues that both model size (N) and dataset size (D) should increase proportionally.
- In Equation (3), a and b have similar values, reflecting a balanced growth strategy [Hoffmann et al., 2022].
- This approach effectively enhances model performance and optimizes resource allocation.

Predictable Scaling

- Predictable scaling refers to reliably estimating the performance of larger models based on smaller models.
- Benefits:
 - For large models, it is impractical to rigorously test various training tricks or variants; insights from small models can be applied.
 - Large-scale models take a long time to train and often face issues like loss spikes; scaling laws can monitor training status.

Predictable Scaling

- **Predictable scaling** refers to reliably estimating the performance of larger models based on smaller models.
- **Benefits:**
 - For large models, it is impractical to rigorously test various training tricks or variants; **insights from small models** can be applied.
 - Large-scale models take a long time to train and often face issues like loss spikes; scaling laws can **monitor training status**.
- **Challenges:**
 - As models grow, public text data for LLMs may soon be exhausted.
 - Exploring data repetition or augmentation can help address data scarcity.

Task-level Predictability

- Existing scaling laws mostly focus on language modeling loss (e.g., per-token cross-entropy loss).
- But task performance is more critical.
- GPT-4 has reported some abilities (e.g., coding ability) that can be predicted by scaling laws.
- However, reducing language modeling loss does not always lead to better task performance; some tasks even degrade.

Task-level Predictability

- Existing scaling laws mostly focus on language modeling loss (e.g., per-token cross-entropy loss).
- But task performance is more critical.
- GPT-4 has reported some abilities (e.g., coding ability) that can be predicted by scaling laws.
- However, reducing language modeling loss does not always lead to better task performance; some tasks even degrade.
- Can all abilities be predicted by Scaling Laws?

Emergent Abilities Behind Scaling Laws

- What happened when the **test loss** decreases **smoothly**?

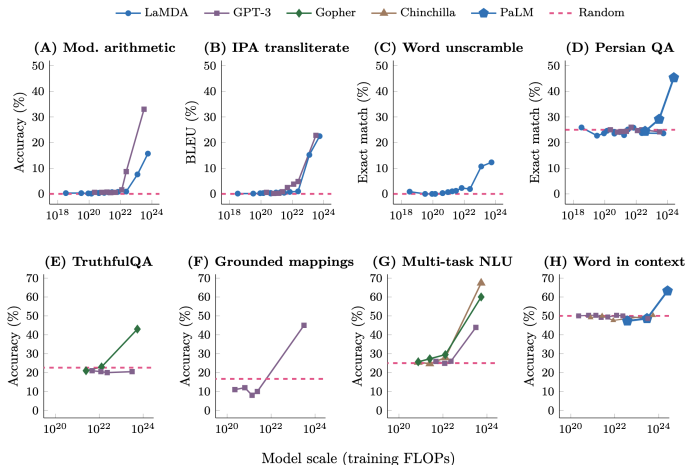


Figure: Examples of emergence in the few-shot prompting setting

What is Emergence?

Emergence defined by Philip Anderson

Emergence is when **quantitative** changes in a system result in qualitative changes in **behavior**

What is Emergence?

Emergence defined by Philip Anderson

Emergence is when **quantitative** changes in a system result in qualitative changes in **behavior**

- A focused definition: An ability is emergent if it is not present in smaller models but is present in larger models. (**Non-smooth**)
- Model display **sharp and unpredictable increases** in performance at specific tasks as scale increases.

Are Emergent Abilities of LLM a Mirage?

- How might **smooth, continuous, predictable** changes in model performance appear **sharp and unpredictable**?

Are Emergent Abilities of LLM a Mirage?

- How might **smooth, continuous, predictable** changes in model performance appear **sharp and unpredictable**?
- The nonlinear or discontinuous metric may lead to the phenomenon [Schaeffer et al., 2024].
- The per-token cross entropy as a function on number of parameters N :

$$\mathcal{L}_{CE}(N) := - \sum_{v \in V} p(v) \log \hat{p}_N(v) \approx - \log \hat{p}_N(v^*)$$

$$p(\text{single token correct}) = \exp(-\mathcal{L}_{CE}(N)) = \exp(-(N/c)^\alpha)$$

$$\text{Accuracy}(N) \approx p_N(\text{single token correct})^L = \exp(-(N/c)^\alpha)^L$$

Smoothness Causes the Disappearance of Emergence

- $\text{Accuracy}(N) \approx \exp(-(N/c)^\alpha)^L$ (**Sharp**)
- $\text{Token Edit Distance}(N) \approx L(1 - \exp(-(N/c)^\alpha))$ (**Smooth**)

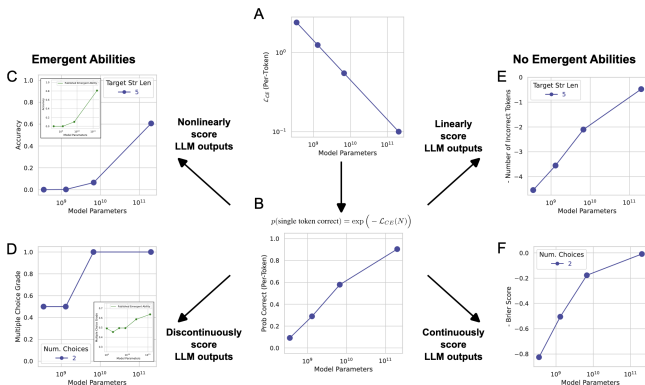


Figure: Emergent abilities seem to disappear

Metrics Must Correspond to the Problem Structure

Intuition

- Consider the addition of two numbers: $10 + 11$.
- However, when considering token edit distance 22 and 11 has the same distance to 21.
- Once we account for the structure of the task, the error in the most significant digit should be penalized more.

Metrics Must Correspond to the Problem Structure

Intuition

- Consider the addition of two numbers: $10 + 11$.
- However, when considering token edit distance 22 and 11 has the same distance to 21.
- Once we account for the structure of the task, the error in the most significant digit should be penalized more.

Experimental validation

- Recent work has demonstrated that even continuous metrics can witness sudden improvements [Chen et al., 2023].
- Emergent capabilities are not likely an artifact of evaluation protocols.

How to Understand Emergence?

- **Emergence** is a well-characterized phenomenon in natural sciences analogous to phase transition.

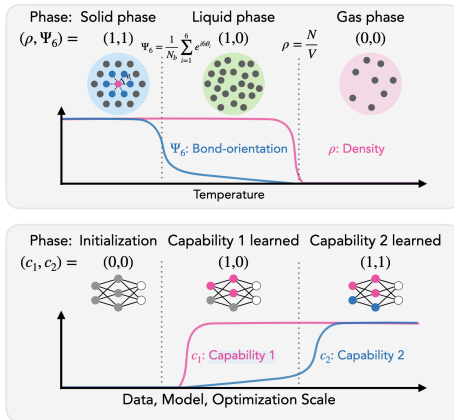


Figure: Emergence phenomenon: Natural Science vs Deep Learning

Percolation Model

- **Percolation model** is one of the simplest tools to describe phase transitions.
- Suppose we immerse a large porous stone in water, what is the probability that the center of the stone is wetted?

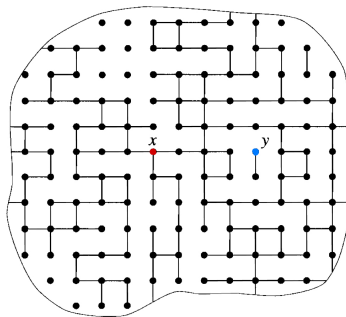
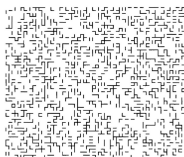


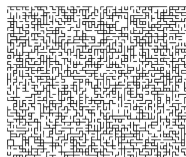
Figure: x will be wetted, but y remain dry

Mathematical Modeling of Percolation

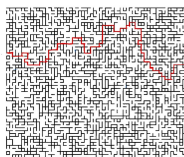
- (Question) Let $\mathbb{Z}^2$ be the plane square lattice and let p be a number satisfying $0 \leq p \leq 1$. We examine each edge of $\mathbb{Z}^2$ in turn, and declare this edge to be open with probability p and closed otherwise, independently of all other edges. Percolation theory is concerned primarily with the existence of infinite connected clusters of open edges.



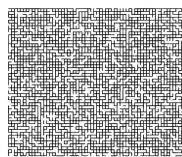
$p = 0.25$



$p = 0.49$



$p = 0.51$



$p = 0.75$

Figure: Realization of bond percolation for different values of p

The Critical Probability Equals 1/2

- The percolation probability is defined as

$$\theta(p) = P\{\text{the origin is part of an infinite connected clusters.}\}$$

- The critical probability p_H as $p_H = \inf\{p : \theta(p) > 0\}$.

Theorem 1

$$p_H = \frac{1}{2}$$



It (percolation theory) is a source of fascinating problems of the best kind a mathematician can wish for: problems which are easy to state with a minimum of preparation, but whose solutions are (apparently) difficult and require new methods.

———— Harry Kesten

Construct a Predictive Learning Theory

- How to define order parameters for studying emergence in neural networks?
- When emergence happens?

Definition 2 (Emergence of a capability)

We say a capability $\mathcal{C}$ is emergent with scaling along a relevant axis if [Lubana et al., 2024]:

- **P1:** nonlinear improvement occurs in the performance of a task where $\mathcal{C}$ is required;
- **P2:** multiple tasks simultaneously show nonlinear performance improvement; and
- **P3:** the model acquires a structure underlying the data generating process, and nonlinear progress in $\mathcal{C}$'s learning directly correlates with the learning of said structure.

Structure I: Grammar

Definition 3 (Grammar)

Consider a set of terminal symbols T and non-terminal symbols NT . A symbolic string σ is defined by $\sigma = t_1 t_2 \cdots \text{EOS}$, $t_i \in T$, and the production rules Q is defined NT , represented as $A \rightarrow \alpha\beta$, where $A \in NT \cup S$ and $\alpha, \beta \in T \cup NT$. The set T^* denotes all possible strings defined using symbols from $T \cup \text{EOS}$. **A probabilistic context-free grammar G is defined as a randomized process** that first maps S to a set of non-terminals according to rules Q , and then repeats the process over these intermediate non-terminals by randomly sampling rules for which previously sampled non-terminals are the LHS. The process continues until only terminal symbols constitute the RHS.

Let $\Sigma \subset T^*$ denote the set of all strings that can be sampled from G . A randomly sampled string $\sigma \in T^*$ is **grammatical** if $\sigma \in \Sigma$.

Structure II: Type Constrains Graph

Definition 4 (Type Constrains Graph)

Let a **property** k be a binary variable; the set of all properties is denoted K . Properties can be either **descriptive** (used to define descriptors, e.g., `tall`) or **relative** (used to define verbs, e.g., `walk`). A **concept class** C is defined via the set $K_C \subset K$ that denotes which properties are valid for that class. When the properties in K_C take values, we get an entity e from the class, denoted as $e \in C$. Entities have unique identifiers associated with them to help define subjects and objects in a sentence. The set of all possible entities is denoted E . The **type constraints graph** $\mathcal{G} := (E, K, I)$ is a bipartite graph over entities and properties in the language whose edges I denote whether an entity $e \in E$ possesses property $k \in K$.

Grammar & Type Constraints to Define Language

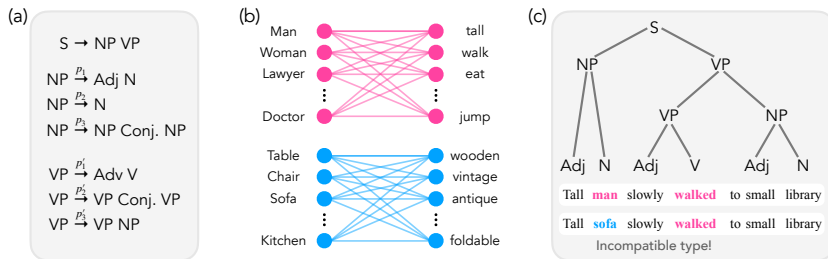


Figure: Grammar and type constraints to define our formal language

Learning Task

- **Free generation:** Produce a valid string, i.e., one that respects the grammar and type constraints.
- **Unscrambling:** A string is sampled from $\mathcal{L}$ and randomly permuted; the model is expected to unscramble it. This task is known to show sudden learning in LLMs.
- **Conditional Generation:** A set of tokens corresponding to entities or properties are shown to the model, which is expected to generate a string combining these tokens in a valid manner.

Learning of Structures Drives Emergence

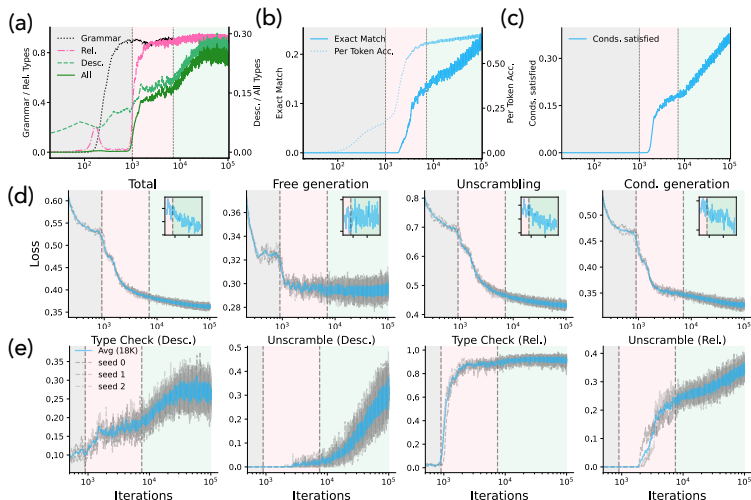


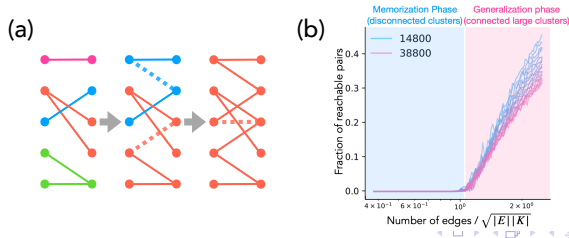
Figure: Grammar and type constraints to define our formal language

Concept Class

Definition 5 (Concept Class)

For the type constraints graph $\mathcal{G} := (E, K, I)$, a Concept Class C is defined via the set $K_C \subset K$ that denotes which properties are valid for one class.

- The corresponding properties for the human concept class will be walk, jump, tall.
- A lawyer, being human, is expected to have all these properties, although it may be rare in training data.



Concept Density Matrix

Definition 6 (Concept Density Matrix)

Let D be an $|E| \times |K|$ matrix with real-valued entries between 0 and 1, inclusive. Each entry D_{ek} represents the density for the entity and property pair (e, k) (e.g., the amount of data that represents the specific composition), where $e \in \{1, \dots, |E|\}$ and $k \in \{1, \dots, |K|\}$ are the indices of the entities and properties, respectively.

For example, consider the case where there are three values of entities and properties ($|E| = |K| = 3$), with entities (rows) being {Man, Lawyer, Telephone}, and properties (columns) being {Walk, Stoic, Ring}. The corresponding D can be:

$$D = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Percolation Threshold

Definition 7 (Concept Propagation Matrix)

An n -th order concept propagation matrix ($n \geq 0$) is defined as $T^{(n)} = (DD^T)^n D = C^n D$, where $C := DD^T$.

- If two concepts are connected by a path of minimal length $2k + 1$, the corresponding entry in $T^{(n)}$ becomes non-zero only for $n \geq k$
- A critical aspect to examine is the proportion of the inference matrix values where $T^{(\infty)}$ is non-zero.

Percolation Threshold

Definition 7 (Concept Propagation Matrix)

An n -th order concept propagation matrix ($n \geq 0$) is defined as $T^{(n)} = (DD^T)^n D = C^n D$, where $C := DD^T$.

- If two concepts are connected by a path of minimal length $2k + 1$, the corresponding entry in $T^{(n)}$ becomes non-zero only for $n \geq k$
- A critical aspect to examine is the proportion of the inference matrix values where $T^{(\infty)}$ is non-zero.
- The percolation threshold is obtained as $p_c \approx \sqrt{1/|E||K|}$.
 - $p < p_c$, the graph typically exhibits a disconnected phase.
 - $p > p_c$, the graph typically exhibits a connected phase.

Outline

- 1 Introduction to Large Language Model
- 2 Scaling Laws and Emergence
- 3 Grokking**
- 4 Chain of Thought
- 5 Advanced Reasoning Model: OpenAI's o1 and o3

Emergence and Grokking Phenomenon

- Generally speaking, emergence is a sudden performance improvement with larger model, more training time and data.
- To understand the emergence phenomenon of LLM, a toy model which can be referred to is “Grokking”.

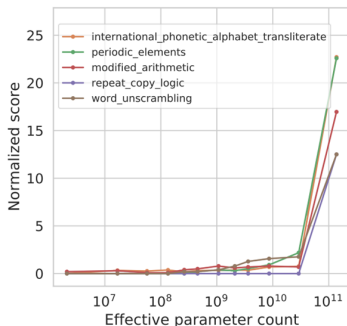
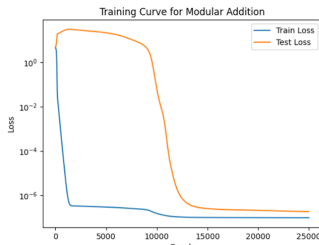


Figure: Emergence of LLM

Grokking: Insights into Generalization and LLMs

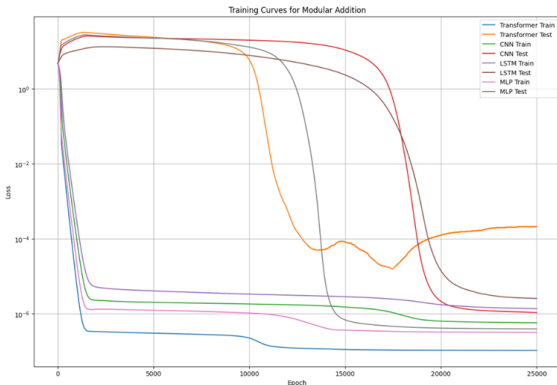
- **Grokking Phenomenon:** Models achieve sudden generalization after extended training beyond initial overfitting.
- **Understanding Generalization:**
 - Reveals transition from memorization to generalization.
 - Highlights critical data size for effective learning.
- **Implications for LLMs:**
 - Informs training strategies to enhance performance.
 - Provides insights into emergent abilities in large models.



Grokking Across Architectures

Observed in Multiple Architectures:

- Recurrent Neural Networks (RNNs)
- Convolutional Neural Networks (CNNs)
- Multi-Layer Perceptrons (MLPs)
- Transformers



Grokking Across Tasks

Manifested in Various Tasks:

- Modular Arithmetic
- MNIST Image Classification
- IMDb Movie Review Sentiment Analysis
- QM9 Molecular Property Prediction

Notably prominent in:

- Tasks with clear regularities, e.g., modular arithmetic

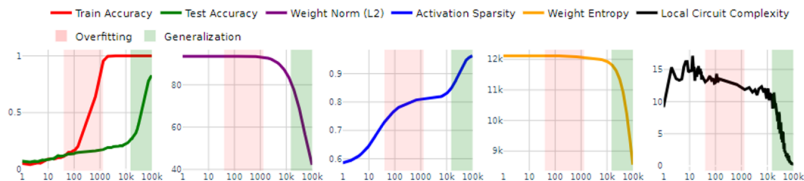


Figure: Grokking; MNIST

Key Factors Influencing Grokking

- **Data-related factors**

- **Dataset Size**: Larger datasets accelerate generalization.
- **Data Distribution**: Imbalanced data can trigger grokking.

- **Initialization**

- **Weight Initialization**: Larger initializations delay grokking.

- **Task Characteristics**

- **Task Regularity**: Simpler patterns are more likely to exhibit grokking.

- **Training Process**

- **Weight Decay**: Encourages efficient generalization (crucial).
- **Training Dynamics**: Affects the transition from fitting to generalizing.

Conclusion

- Language modeling performance improves smoothly as we increase the models size (scaling-Law).

Conclusion

- Language modeling performance improves smoothly as we increase the models size (scaling-Law).
- **Emergence** is a sudden performance improvement not present in small models.

Conclusion

- Language modeling performance improves smoothly as we increase the models size (scaling-Law).
- **Emergence** is a sudden performance improvement not present in small models.
- To understand the emergence phenomenon of LLM, a toy model which can be referred to is **Grokking**.
 - Models achieve sudden generalization after **extended traing** beyond initial overfitting.
- The theory related to emergence of LLMs is primarily **demonstrative**.
- How to **explain and predict** emergent capabilities in neural networks still remains an open problem.

Outline

- 1 Introduction to Large Language Model
- 2 Scaling Laws and Emergence
- 3 Grokking
- 4 Chain of Thought**
- 5 Advanced Reasoning Model: OpenAI's o1 and o3

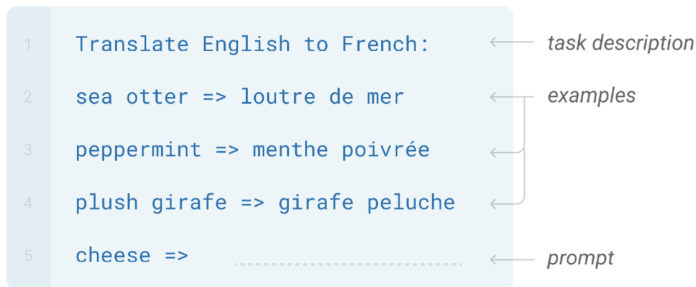
Capabilities of LLMs

Large Language Models (LLMs) have demonstrated emergent capabilities in various aspects:

- Generation: translation, summary, composition, . . .
- Question answering
- Mathematics
- Coding
- Reasoning, planning, decision-making, . . .

Autoregressive Transformers

- Most LLMs follow the **autoregressive design** paradigm.
- **Main idea**: Various tasks can be uniformly treated as **sequence generation problems**.
- The task description can be together encoded as tokens, called the **prompt**.
- The answer is generated by predicting subsequent tokens **conditioned on the prompt** in an autoregressive way.



LLMs Fail in Some Elementary Math Tasks

LLMs may even fail to perform compound operations.

- Below I will ask you a few questions. When a question is prefixed with [Direct], please directly generate the answer without giving any intermediate steps. On the contrary, when a question is prefixed with [Chain], please answer the question in details step by step, and give the final answer in the end.

Okay, I'm ready. Please provide the questions with the appropriate prefixes.

首token 0.71s | 平均 12 tokens/s | 总耗时 1.3s | 共调用 488 tokens

[Direct] $3+(7-(7-2)*(5-3))-(6-4*(1+2))=?$

9

首token 0.41s | 平均 2 tokens/s | 总耗时 0.42s | 共调用 525 tokens

Figure: The result showed by Doubao-pro-32k-241215

LLMs Success When Using CoT

However, when we use CoT...

🟡 [Chain] $3 + (7 - (7 - 2) \times (5 - 3)) - (6 - 4 \times (1 + 2)) = ?$

We will solve this expression step by step according to the order of operations (PEMDAS: Parentheses, Exponents, Multiplication and Division, Addition and Subtraction).

Step 1: Solve the innermost parentheses

- In the expression $3 + (7 - (7 - 2) \times (5 - 3)) - (6 - 4 \times (1 + 2))$, we first solve the expressions inside the innermost parentheses:
 - For $(7 - 2)$, we have $7 - 2 = 5$
 - For $(5 - 3)$, we have $5 - 3 = 2$
 - For $(1 + 2)$, we have $1 + 2 = 3$

The expression now becomes: $3 + (7 - 5 \times 2) - (6 - 4 \times 3)$

Step 2: Perform the multiplications

- For 5×2 , we have $5 \times 2 = 10$
- For 4×3 , we have $4 \times 3 = 12$

The expression now becomes: $3 + (7 - 10) - (6 - 12)$

Step 3: Solve the remaining parentheses

- For $(7 - 10)$, we have $7 - 10 = -3$
- For $(6 - 12)$, we have $6 - 12 = -6$

The expression now becomes: $3 + (-3) - (-6)$

Step 4: Simplify the expression with addition and subtraction

- $3 + (-3) = 3 - 3 = 0$
- Then $0 - (-6) = 0 + 6 = 6$

So, $3 + (7 - (7 - 2) \times (5 - 3)) - (6 - 4 \times (1 + 2)) = 6$

• With CoT, Doubao can get the right result!

Questions Regarding CoT

How to theoretically understand the power of CoT?

- How can these prompts trigger the CoT generalization?
- How to design better prompting strategies to improve the power of LLMs?

Questions Regarding CoT

How to theoretically understand the power of CoT?

- How can these prompts trigger the CoT generalization?
- How to design better prompting strategies to improve the power of LLMs?

Two Example Tasks: Arithmetic and Equation

- Can we understand the CoT power in fundamental math tasks?
- **Arithmetic and equation** serve as elementary building blocks in solving complex real-world math problems [Feng et al., 2024].

Questions Regarding CoT

How to theoretically understand the power of CoT?

- How can these prompts trigger the CoT generalization?
- How to design better prompting strategies to improve the power of LLMs?

Two Example Tasks: Arithmetic and Equation

- Can we understand the CoT power in fundamental math tasks?
- **Arithmetic and equation** serve as elementary building blocks in solving complex real-world math problems [Feng et al., 2024].
- **Arithmetic** (n, p): the task of evaluating arithmetic expressions (modulo p), where the input length is bounded by n .
- **Equation** (m, p): the task of solving linear equations (modulo p) with no more than m variables.

The Natural CoTs Exist

- Both problems have natural CoT solutions.
 - Arithmetic**: reducing one atomic operation at a time while copying down others.
 - Equation**: Gaussian elimination algorithm.

Arithmetic Expression	Linear Equations
Input: $(7 + 5) \div (6 + 4 \times 3 - 2 \times 7) =$ Output: $\begin{aligned} &12 \div (6 + 4 \times 3 - 2 \times 7) \\ &= 12 \div (6 + 12 - 2 \times 7) \\ &= 12 \div (18 - 2 \times 7) \\ &= 12 \div (18 - 14) \\ &= 12 \div 4 \\ &= 3 \end{aligned}$	Input: $\begin{aligned} 3x + 3y + 12z &= 6; \\ 2x + 5y + 14z &= 7; \\ 2x + 4y + 15z &= 6; \end{aligned}$ $\Rightarrow$ Output: $\begin{aligned} x + 1y + 4z &= 2; \\ 3y + 6z &= 3; \\ 2y + 7z &= 2; \end{aligned}$ $\Rightarrow \begin{aligned} x + 2z &= 1; \\ y + 2z &= 1; \\ 3z &= 0; \end{aligned}$ $\Rightarrow \begin{aligned} x &= 1; \\ y &= 1; \\ z &= 0; \end{aligned}$

Log-precision Transformer

- Multi-layer perceptrons with sufficient size are already universal function approximators [Cybenko, 1989, Leshno et al., 1993].
- Recently, such results have been well extended to Transformer models [Yun et al., 2019].

Log-precision Transformer

- Multi-layer perceptrons with sufficient size are already universal function approximators [Cybenko, 1989, Leshno et al., 1993].
- Recently, such results have been well extended to Transformer models [Yun et al., 2019].
- We consider a **practical setting** called the log-precision Transformer whose internal neurons can only store **floating-point numbers** within a finite $O(\log n)$ bit precision.
- Example: 16/32 bits machine precision v.s. maximal sentence length of 2048 in GPT.
- Why log-precision? It implies that the number of values each neuron can take is **polynomial** in the input length.

Negative Results

Theorem 8

Assume $TC^0 \neq NC^1$. For any prime number p , integer L , and any polynomial Q ,

- there exists a problem size n such that *no log-precision autoregressive Transformer* with depth L and hidden dimension $d \leq Q(n)$ can directly solve the problem $Arithmetic(n, p)$
 - there exists a problem size m such that *no log-precision autoregressive Transformer* with depth L and hidden dimension $d \leq Q(m)$ can directly solve the problem $Equation(m, p)$.
-
- The theorem implies that in order to directly output the answers, the size of model will grow super-polynomially in the input length for both problems.

Circuit Complexity

Definition 9 (Boolean circuit [Arora and Barak, 2009])

For every $n, m \in \mathbb{N}$ a Boolean circuit C with n input nodes and m output nodes is a directed acyclic graph. All other nodes are called gates and are labeled with one of $\wedge, \vee$ or $\neg$. The size of C denoted by $|C|$ is the number of nodes in it.

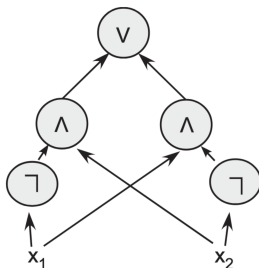


Figure: A circuit C computing the XOR function

NC, AC and TC Complexity

Definition 10 (NC Complexity)

A problem $\mathcal{L}$ is in NC^k if and only if there exists a polynomial $p(n)$ and a family of circuits $\{C_n\}$ such that each circuit C_n has n inputs and 1 output, the fan-in of the gates is at most 2, $|C_n|$ is at most $p(n)$, the depth of each circuit C_n is $O((\log n)^k)$ and for all strings x , $x \in \mathcal{L}$ if and only if $C_{|x|}(x) = 1$.

- The class AC^k is defined almost the same as NC^k except the $\wedge$ and $\vee$ gates in AC^k allow unbounded fan-in.
- The class TC^k allows a more powerful type of gate, MAJORITY, compared to AC^k ,

$$\text{MAJORITY}(x_1, \dots, x_n) = \left\lfloor \frac{1}{2} + \frac{(\sum_{i=1}^n x_i) - \frac{1}{2}}{n} \right\rfloor.$$

Key Insight into the Proof: Circuit Complexity

- TC^0 and NC^1 are two standard computation complexity classes, and it is widely believed that $TC^0 \not\subseteq NC^1$.

$$TC^0 \subset NC^1 \subset P \subset NP$$

Key Insight into the Proof: Circuit Complexity

- TC^0 and NC^1 are two standard computation complexity classes, and it is widely believed that $TC^0 \not\subseteq NC^1$.

$$TC^0 \subset NC^1 \subset P \subset NP$$

- A pure Transformer represents a class of shallow circuits with complexity **upper bounded by TC^0** .
- On the other hand, the complexity of both math problems above are **lower bounded by NC^1** .

How About Generating a CoT Solution?

Theorem 11

*Fix any prime p . For any integer $n > 0$, there exists an autoregressive Transformer with constant hidden size d (*independent of n*), depth $L = 5$, and 5 heads in each layer that can generate the CoT solution for all inputs in Arithmetic (n, p) . Moreover, all parameter values in the Transformer are bounded by $O(\text{poly}(n))$.*

Theorem 12

*Fix any prime p . For any integer $m > 0$, there exists an autoregressive Transformer with constant hidden size d (*independent of m*), depth $L = 5$, and 5 heads in each layer that can generate the CoT solution for all inputs in Equation (m, p) . Moreover, all parameter values in the Transformer are bounded by $O(\text{poly}(m))$.*

Key Insight into the Proof

- The proof reveals the significance of several key components in the Transformer design:
 - One attention head can perform the following two basic operations: copy and reduction.
 - Multi-head attention can perform multiple copy/reduction operation in parallel.
 - The MLP can perform multiplication, linear transformation, conditional selection, and look-up table.
 - Residual connection can reserve the history information.
- CoT can enable the model to store the intermediate results.

CoT Can Implement Dynamic Programming

- LLMs with CoT are theoretically capable of emulating a powerful decision-making framework called **Dynamic Programming** in the following format

input 1 | $\dots$ | input n | $(i_1, \text{dp}(i_1)) \dots (i_n, \text{dp}(i_n))$ | final answer

Theorem 13

Informal Consider a DP problem satisfying some regularity assumptions. For any integer $n \in \mathbb{N}$, there exists an autoregressive Transformer with constant depth L , hidden dimension d and attention heads H (independent of n), such that the answer generated by the Transformer is correct for all input sequences s of length no more than n . Moreover, all parameter values are bounded by $O(\text{poly}(n))$.

Impossibility Results

- Many DP problems are intrinsically hard to be solved by a bounded-depth Transformer without CoT.
- One celebrate example is the Context-Free Grammer (CFG) Membership Testing, which tests whether an input string belongs to a pre-defined context-free language.

Impossibility Results

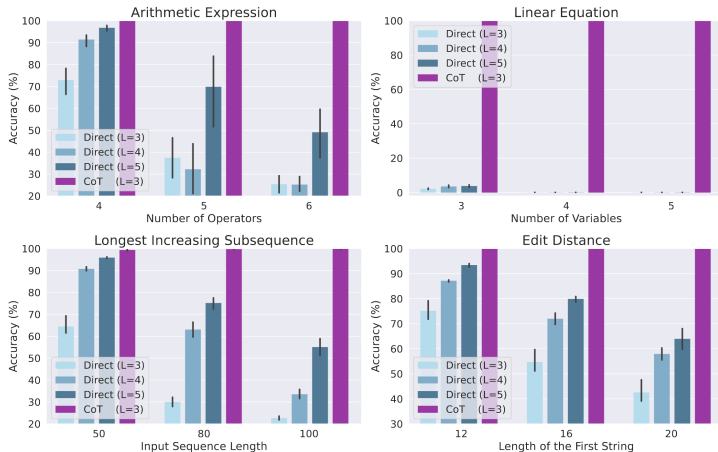
- Many DP problems are intrinsically hard to be solved by a bounded-depth Transformer without CoT.
- One celebrate example is the Context-Free Grammer (CFG) Membership Testing, which tests whether an input string belongs to a pre-defined context-free language.
- The theorem below shows that the Transformers without CoT **CANNOT** solve CFG Membership Testing Problem.

Theorem 14

*Assume $TC^0 \neq P$. There exists a **context-free** language such that for any depth L and any polynomial Q , there exists a sequence length $n \in \mathbb{N}$ where no log-precision autoregressive transformer with depth L and hidden dimension $d \leq Q(n)$ can generate the correct answer for the CFG Membership Testing Problem for all input strings of length n .*

Experiments

- Autoregressive Transformers with CoT consistently outperform Transformers trained on direct datasets.



Limitation of Transformer Architecture

- Transformers are not designed to perform reasoning.

Limitation of Transformer Architecture

- Transformers are not designed to perform reasoning.
- **Example: Simulate a chess game.**
 - Denote the sequence of chess moves $\mathbf{x}_n = (x_1, x_2, \dots, x_n)$.
 - To validate the n -th move, the n -th board state $\mathbf{h}_n$ must be calculated.

$$x_n \rightarrow h_n \rightarrow x_{n+1}$$

Limitation of Transformer Architecture

- Transformers are not designed to perform reasoning.
- **Example: Simulate a chess game.**
 - Denote the sequence of chess moves $\mathbf{x}_n = (x_1, x_2, \dots, x_n)$.
 - To validate the n -th move, the n -th board state $\mathbf{h}_n$ must be calculated.

$$x_n \rightarrow h_n \rightarrow x_{n+1}$$

- From h_n to x_{n+1} is easy due to sparsity.
- For $x_n \rightarrow h_n$, while a network could memorize the mapping from x_n to the correct h_n , memorization is much more resource-intensive than reasoning.

Limitation of Transformer Architecture

- Transformers are not designed to perform reasoning.
- **Example: Simulate a chess game.**
 - Denote the sequence of chess moves $\mathbf{x}_n = (x_1, x_2, \dots, x_n)$.
 - To validate the n -th move, the n -th board state $\mathbf{h}_n$ must be calculated.

$$x_n \rightarrow h_n \rightarrow x_{n+1}$$

- From h_n to x_{n+1} is easy due to sparsity.
- For $x_n \rightarrow h_n$, while a network could memorize the mapping from x_n to the correct h_n , memorization is much more resource-intensive than reasoning.
- In transformer model, the hidden state h is passed forward only through the **layers** of the Transformer, not through **step**.

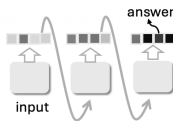
CoT + Autoregressive = Recurrent

- CoT generates intermediate steps as tokens ($o_1, o_2, \dots, o_k$), act as a discretization of the latent information h .

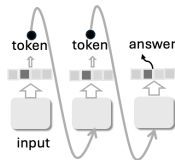
$$h_t \rightarrow (o_1, \dots, o_k) \rightarrow h_{t+1}$$

- Thus, Chain of Thought (CoT) is crucial for tasks involving reasoning.

(a) Recurrent Network



(b) Autoregressive Network



(c) Chain of Thought

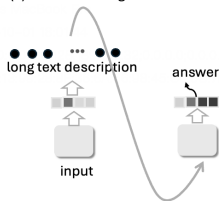


Figure: Comparison between recurrence and autoregression

One Prompt For All Is Not Good Enough

- **One prompt for all approach**, such as “think step by step”, is imperfect.
- When LLMs are prompted to “think step by step”, they generate a step template.

One Prompt For All Is Not Good Enough

- **One prompt for all approach**, such as “think step by step”, is imperfect.
- When LLMs are prompted to “think step by step”, they generate a step template.
- If we define the amount of information stored in h as m bits, and each CoT step extracts up to s bits of information into o .
- Thus, the total number of potential step templates is
$$C(m, s) = \frac{m!}{s!(m-s)!}.$$
- If the step template is not appropriate, **the information extraction may be inefficient!**

Exploring Multiple Paths in Answer Space

- In practice, the **answer space** S can be large and complex.
- Various CoT variants (e.g., Tree-of-Thought (ToT) [Yao et al., 2024] and Graph-of-Thought (GoT) [Besta et al., 2024]) have been proposed to mitigate mistakes in solution searching.

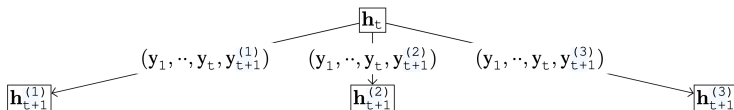


Figure 4: ToT mechanism. h_t is transitioned into different h_{t+1} , to explore more in *answer space*. How state is transitioned is dictated by the step template of CoT, which goes beyond what ToT offers.

Figure: ToT mechanism.

CoT Space = Prompt Space + Answer Space

- The navigation of the prompt space can simplify the answer space complexity and reveal limitations of unsupervised “X-of-thought” methods.

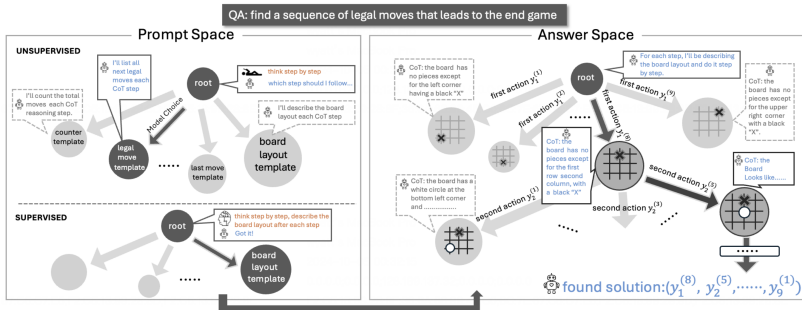


Figure: A visualization of CoT search space.

Experiment

- **Recurrence** is key for reasoning.
- **Supervision CoT** is essential.

Level	Task	RNN	Tape RNN	Transformer	LLM w/o CoT	CoT Unsupervised	CoT CR Supervised	CoT IN Supervised
R	Modular Arithmetic	1.00	1.00	0.24	0.22	0.96	1.00	0.44
	Parity Check	1.00	1.00	0.52	0.58	0.94	1.00	0.42
	Cycle Navigation	1.00	1.00	0.62	0.50	0.78	1.00	0.26
CF	Stack Manipulation	0.56	1.00	0.58	0.00	0.92	0.96	0.00
	Reverse List	0.62	1.00	0.62	0.00	0.80	0.96	0.38
	Modular Arithmetic	0.41	0.95	0.32	0.00	0.82	0.94	0.50
CS	Odds First	0.51	1.00	0.53	0.00	0.80	0.92	0.00
	Addition	0.50	1.00	0.54	0.00	0.84	0.88	0.00
	Multiplication	0.50	0.59	0.52	0.00	0.14	0.44	0.00
	Sorting	0.28	0.71	0.92	0.00	0.36	0.90	0.00

- **Note.** CR Supervised means CoT under the correct supervision, while IN Supervised means suboptimal supervision.

Conclusion

- **Transformer** is the core architecture of most mainstream LLMs.
- However, it has inherent limitations in computational depth, rendering them theoretically incapable of solving many **reasoning tasks** that demand increasingly deep computations.

Conclusion

- **Transformer** is the core architecture of most mainstream LLMs.
- However, it has inherent limitations in computational depth, rendering them theoretically incapable of solving many **reasoning tasks** that demand increasingly deep computations.
- **Chain of Thought (CoT)** prompting has emerged as a technique to address these architectural limitations.

Conclusion

- **Transformer** is the core architecture of most mainstream LLMs.
- However, it has inherent limitations in computational depth, rendering them theoretically incapable of solving many **reasoning tasks** that demand increasingly deep computations.
- **Chain of Thought (CoT)** prompting has emerged as a technique to address these architectural limitations.
- However, how the CoT generation process is triggered by specific prompts remains unclear.
- Revealing the relation between CoT prompts and outputs is valuable for better **understanding** and **harnessing** LLMs

Outline

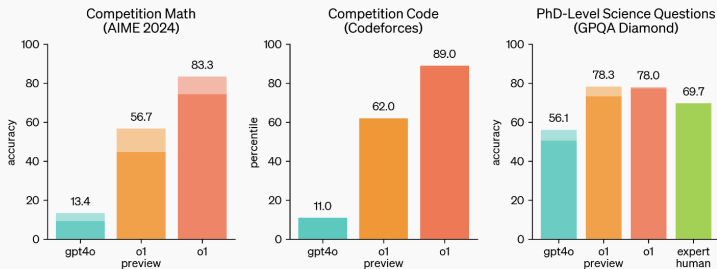
- 1 Introduction to Large Language Model
- 2 Scaling Laws and Emergence
- 3 Grokking
- 4 Chain of Thought
- 5 Advanced Reasoning Model: OpenAI's o1 and o3

Overview of OpenAI o1

- **Advanced Reasoning Model**: o1 is designed to spend more time "thinking" before responding, enhancing its ability to handle complex reasoning tasks, particularly in science and mathematics.
- **Variants**: Available in two versions: o1 and o1-mini. o1-mini is faster and 80% cheaper, suitable for programming and STEM-related tasks.
- **Availability**: Accessible to ChatGPT Plus and Team users, with plans for broader access in the future.

o1 Performance Highlights

- Ranks in the 89th percentile on competitive programming questions (Codeforces) [OpenAI, 2024].
- Among the top 500 students in the US in a qualifier for the USA Math Olympiad (AIME).
- Exceeds human PhD-level accuracy on a benchmark of physics, biology, and chemistry problems (GPQA).



o1 Performance Highlights

Dataset	Metric	gpt-4o	o1-preview	o1
Competition Math AIME (2024)	cons@64	13.4	56.7	83.3
	pass@1	9.3	44.6	74.4
Competition Code CodeForces	Elo	808	1,258	1,673
	Percentile	11.0	62.0	89.0
GPQA Diamond	cons@64	56.1	78.3	78.0
	pass@1	50.6	73.3	77.3
Biology	cons@64	63.2	73.7	68.4
	pass@1	61.6	65.9	69.2
Chemistry	cons@64	43.0	60.2	65.6
	pass@1	40.2	59.9	64.7
Physics	cons@64	68.6	89.5	94.2
	pass@1	59.5	89.4	92.8
MATH	pass@1	60.3	85.5	94.8
MMLU	pass@1	88.0	90.8	92.3
MMMU (val)	pass@1	69.1	n/a	78.2
MathVista (testmini)	pass@1	63.8	n/a	73.9

Figure: Performance in Different Tasks

Example

- Prompt: Five people (A, B, C, D, and E) are in a room. A is watching TV with B, D is sleeping, B is eating chow min, and E is playing table tennis. Suddenly, a call comes on the telephone. B goes out of the room to pick up the call. What is C doing?

There are five people (A, B, C, D, and E) in a room. A is watching TV with B, D is sleeping, B is eating chow min, and E is playing table tennis. Suddenly, a call comes on the telephone. B goes out of the room to pick up the call. What is C doing?



C is not mentioned as doing anything in the description, so based on the information provided, we don't know what C is doing. This seems like a classic riddle where C is likely the one telling the story or just observing the others.

🔊 📄 👍 🗑️ ↺

Figure: Answer of GPT-4o

Example

There are five people (A, B, C, D, and E) in a room. A is watching TV with B, D is sleeping, B is eating chow min, and E is playing table tennis. Suddenly, a call comes on the telephone. B goes out of the room to pick up the call. What is C doing?



Thought for 8 seconds ^

Providing details

I'm noting the number of words and letters in Strawberry and detailing the individuals' activities in a room scenario.

Determining C's activity

Given that B is watching TV, eating chow mein, and answering the call, it raises the question: what is C doing?

Revisiting the scene

I'm unraveling the details: B's departure left A alone in front of the TV, while E played table tennis, possibly with C. D, meanwhile, was still asleep.

Observing the dynamics

I'm noticing E engaging in table tennis with another person, as it requires a partner. C is logically the person with E in this scenario.

C is playing table tennis with E.

Since E is playing table tennis and table tennis typically requires two players, and C is the only person not accounted for, it follows that C is playing with E. •

Figure: Answer of o1

o1 Performance Highlights

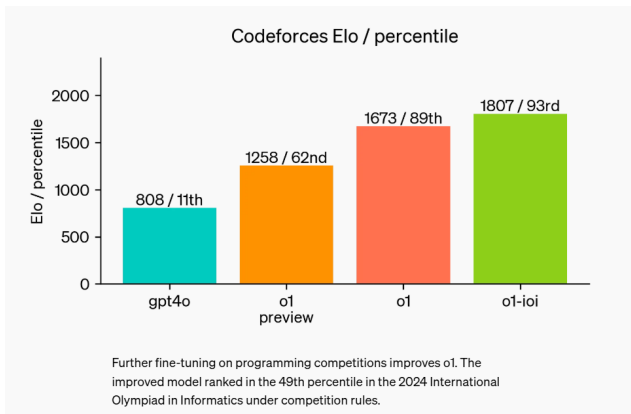


Figure: Performance Improvement in Codeforces Elo and Percentile Rankings through Model Fine-Tuning

Training Methods

- Trained with a large-scale reinforcement learning algorithm.
- Thinks productively using its chain of thought in a highly data-efficient process.
- Performance improves with more reinforcement learning and more thinking time.

Human Preference Evaluation

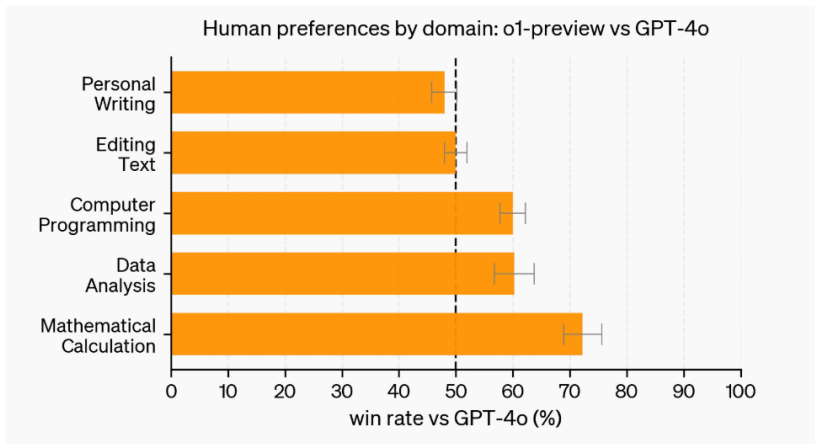


Figure: Human Preferences by Domain

Self-Reflection and Error Correction

- o1 introduces self-reflection and error correction to large models.
- Unlike models like GPT-4, o1 can recognize and correct its own mistakes during extended token output.
- This capability is crucial for long-chain thinking and complex problem-solving.

Scaling Laws and Tree Search Structures

- o1 likely employs advanced tree search structures, similar to AlphaGo's MCTS.
- This approach enhances logical reasoning and problem-solving abilities.
- The flexibility of parameter configuration allows for scalable performance improvements.

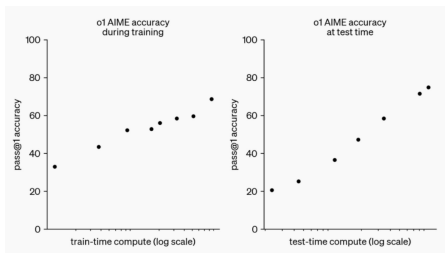


Figure: o1 RL Train-time Scaling vs o1 Inference-time Scaling

Potential for Smaller Models and Safety Alignment

- o1's advancements make it possible for smaller models to achieve capabilities previously locked to larger models.
- The model's enhanced logical reasoning can lead to new paradigms in safety alignment.
- "Capability Divide and Conquer" (DCA) could become a new norm, allowing smaller models to excel in specific domains.

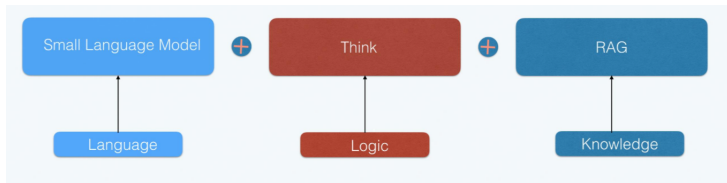


Figure: Divide-and-Conquer of Ability

Generalization Beyond STEM Fields

- o1's reinforcement learning and logical reasoning could extend to fields beyond STEM (Science, Technology, Engineering, Mathematics).
- The potential for defining rewards in non-quantitative fields could broaden o1's applicability.
- This could lead to a new era of AI development, where models are trained to understand and follow complex, rule-based guidelines.

Comparison between o3 and o1

o1 Model

- Focused on foundational AI tasks.
- Utilized standard training methodologies.
- Achieved significant milestones in natural language processing.

Comparison between o3 and o1

o1 Model

- Focused on foundational AI tasks.
- Utilized standard training methodologies.
- Achieved significant milestones in natural language processing.

o3 Model

- Represents a significant advancement over o1.
- Introduces enhanced reasoning abilities and deliberative alignment.
- Achieved superior performance across various complex tasks.

Advances in Engineering and Coding

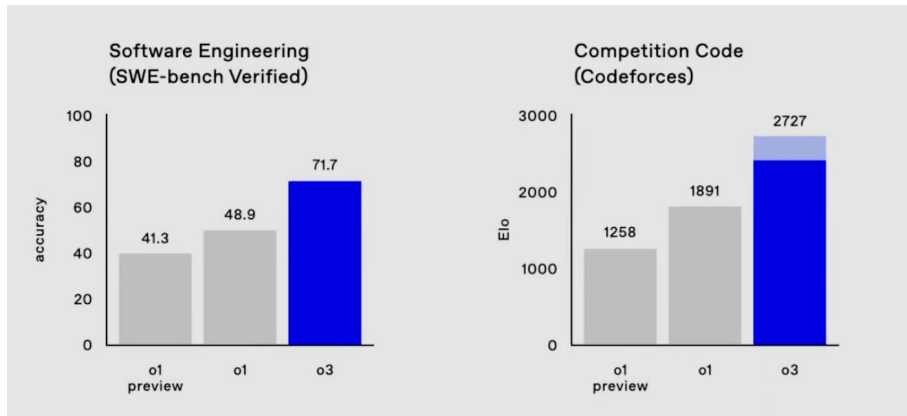


Figure: Comparison between o3 and o1 in Engineering and Coding

Advances in Maths and Science

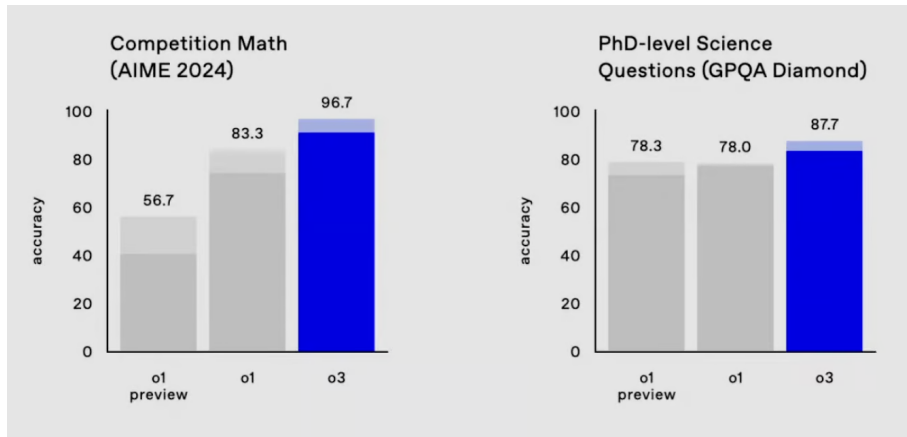


Figure: Comparison between o3 and o1 in Maths and Science

Overview of EpochAI FrontierMath Benchmark

- **Highly Challenging Benchmark:** Recognized as one of the most demanding in AI.
- **Novel, Unpublished Problems:** Contains problems intentionally designed to be significantly more difficult than standard datasets.
- **Research-Level Mathematics:** Many problems are at the level of mathematical research, often requiring professional mathematicians hours or days to solve.
- **AI Performance:** Current AI systems typically score below 2% on this benchmark, highlighting its difficulty.

o3 Performance on EpochAI FrontierMath Benchmark

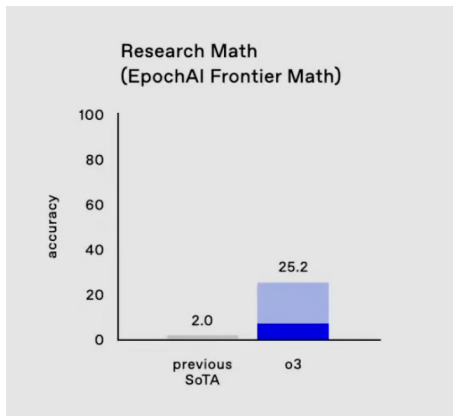


Figure: Performance on EpochAI FrontierMath Benchmark

Abstraction and Reasoning Corpus (ARC)

- Developed by François Chollet in 2019.
- Assesses AI's ability to learn and generalize from minimal examples.
- Challenges models with dynamic rule inference and pattern transformation.
- Requires different reasoning skills for each task, preventing reliance on memorization.

Breaking New Ground

- Low compute setting: 76% on semi-private holdout set.
- High compute setting: 88%, surpassing the 85% human-level threshold.
- First AI to exceed human performance on this benchmark.

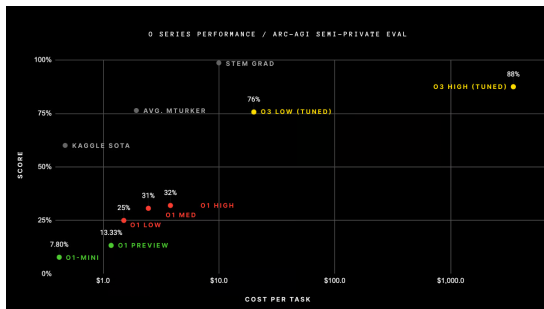


Figure: o3's Performance on ARC AGI

Towards True General Intelligence

- o3 demonstrates adaptability and generalization over rote memorization.
- Indicates progress towards general intelligence, transcending domain-specific skills.
- Enters realms previously considered exclusive to human cognition.

Innovative Safety Strategy for o3

- OpenAI's proactive approach: Public safety evaluations before full model release.
- Deliberative alignment: A method surpassing conventional safety methods.
- Distinguishes from RLHF, RLAIIF, and Self-REFINE by using real-time reasoning.

Innovative Safety Strategy for o3

- OpenAI's proactive approach: Public safety evaluations before full model release.
- Deliberative alignment: A method surpassing conventional safety methods.
- Distinguishes from RLHF, RLAIIF, and Self-REFINE by using real-time reasoning.

Training Data Generation

- Utilizes a reasoning model to generate CoT outputs for prompts.
- These outputs guide training, enhancing context and intent understanding.

Inference Time

- The model evaluates prompts and provides CoT explanations with answers.

References I



Arora, S. and Barak, B. (2009).
Computational complexity: a modern approach.
Cambridge University Press.



Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Podstawski, M., Gianinazzi, L., Gajda, J., Lehmann, T., Niewiadomski, H., Nyczyk, P., et al. (2024).
Graph of thoughts: Solving elaborate problems with large language models.
In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 17682–17690.



Chen, A., Shwartz-Ziv, R., Cho, K., Leavitt, M. L., and Saphra, N. (2023).
Sudden drops in the loss: Syntax acquisition, phase transitions, and simplicity bias in mlms.
arXiv preprint arXiv:2309.07311.



Cybenko, G. (1989).
Approximation by superpositions of a sigmoidal function.
Mathematics of control, signals and systems, 2(4):303–314.



Feng, G., Zhang, B., Gu, Y., Ye, H., He, D., and Wang, L. (2024).
Towards revealing the mystery behind chain of thought: a theoretical perspective.
Advances in Neural Information Processing Systems, 36.

References II



Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J. W., Vinyals, O., and Sifre, L. (2022).

Training compute-optimal large language models.

arXiv preprint, [abs/2203.15556](https://arxiv.org/abs/2203.15556).



Leshno, M., Lin, V. Y., Pinkus, A., and Schocken, S. (1993).

Multilayer feedforward networks with a nonpolynomial activation function can approximate any function.

Neural networks, 6(6):861–867.



Lubana, E. S., Kawaguchi, K., Dick, R. P., and Tanaka, H. (2024).

A percolation model of emergence: Analyzing transformers trained on a formal language.

arXiv preprint [arXiv:2408.12578](https://arxiv.org/abs/2408.12578).



Neyshabur, B., Li, Z., Bhojanapalli, S., LeCun, Y., and Srebro, N. (2018).

Towards understanding the role of over-parametrization in generalization of neural networks.

arXiv preprint [arXiv:1805.12076](https://arxiv.org/abs/1805.12076).



OpenAI (2020).

Scaling laws for neural language models.

arXiv preprint [arXiv:2001.08361](https://arxiv.org/abs/2001.08361).

References III



OpenAI (2024).
Learning to reason with llms.



Schaeffer, R., Miranda, B., and Koyejo, S. (2024).
Are emergent abilities of large language models a mirage?
Advances in Neural Information Processing Systems, 36.



Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. (2024).
Tree of thoughts: Deliberate problem solving with large language models.
Advances in Neural Information Processing Systems, 36.



Yun, C., Bhojanapalli, S., Rawat, A. S., Reddi, S. J., and Kumar, S. (2019).
Are transformers universal approximators of sequence-to-sequence functions?
arXiv preprint arXiv:1912.10077.



Zhang, S., Dong, L., Li, X., Zhang, S., Sun, X., Wang, S., Li, J., Hu, R., Zhang, T., Wu, F., et al. (2023).
Instruction tuning for large language models: A survey.
arXiv preprint arXiv:2308.10792.



Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Liu, P., Nie, J.-Y., and Wen, J.-R. (2024).
A survey of large language models.